Circuit Breaker

Season: 4 · Enterprise · Category: Enterprise · Level: Intermediate

Intent

- Fail fast when a dependency is down.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Electrical breaker trips to protect the house.

When to Use

- Microservices / modular monoliths.
- The design force named in Intent shows up more than once.

When Not to Use

- Greenfield CRUD without distribution.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.enterprise.circuitbreaker;

import java.time.Duration;
import java.time.Instant;
import java.util.concurrent.Callable;
import java.util.function.Supplier;

public final class CircuitBreaker {
    public enum State { CLOSED, OPEN, HALF_OPEN }

    private final int failureThreshold;
    private final Duration openTimeout;
    private final Supplier<Instant> clock;

    private State state = State.CLOSED;
    private int failures;
    private Instant openedAt = Instant.EPOCH;

    public CircuitBreaker(int failureThreshold, Duration openTimeout, Supplier<Instant> clock) {
        this.failureThreshold = failureThreshold;
        this.openTimeout = openTimeout;
        this.clock = clock;
    }

    public State state() { return state; }

    public <T> T call(Callable<T> action) throws Exception {
        if (state == State.OPEN) {
            if (clock.get().isAfter(openedAt.plus(openTimeout))) {
                state = State.HALF_OPEN;
            } else {
                throw new IllegalStateException("circuit open");
            }
```

Classic Implementation (2)

```
            }
            try {
                var result = action.call();
                failures = 0;
                state = State.CLOSED;
                return result;
            } catch (Exception e) {
                failures++;
                if (failures >= failureThreshold || state == State.HALF_OPEN) {
                    state = State.OPEN;
                    openedAt = clock.get();
                }
                throw e;
            }
        }
}
```

Modern Java 25 / Functional

- Sealed events + pattern matching for projections.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: enterprise/circuitbreaker/
```

Pitfalls

- Keep demos minimal — production needs idempotency and observability.
- Saga compensations must be careful.

Interview Q&A

- Q: Production concern for Circuit Breaker? A: Idempotency, observability, and failure modes.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Fail fast when a dependency is down.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Electrical breaker trips to protect the house. [0:40] Intent — State the design force in one sentenc...